

MLOps Lecture: A Simple Kubernetes Intro Demo with a MongoDB Web App

AI 545 / MLOps

1 Lecture Goal

In this lecture, we will build a very simple Kubernetes application locally on macOS using Minikube and Docker Desktop. The goal is to introduce Kubernetes through a concrete web application rather than through abstract theory alone. Our application will contain two parts. The first part will be a very small Python web app. The second part will be a MongoDB database. The web app will show a single HTML form with one text field. When the user submits the form, the app will store the latest value in MongoDB. We will then deploy both pieces with Kubernetes and access the web application locally.

This example is intentionally small. The purpose is not to build a sophisticated production system, but to understand the mechanics of Kubernetes: how to package an application into a container, how to define Deployments and Services, how to provide configuration and secrets, and how to persist database data.

2 Why This Demo Matters in MLOps

Even though this is not yet a full machine learning system, the architecture is already very similar to what appears in MLOps. In a real MLOps platform, a model-serving API might store metadata, predictions, logs, features, or experiment information in a database. The web app in this lecture stands in for a small API or dashboard, and MongoDB stands in for the database layer. By learning this simpler application first, students can later understand how the same Kubernetes ideas extend to inference services, model registries, feature stores, and experiment tracking systems.

3 Local Environment Assumptions

We assume that Docker Desktop is installed and running on macOS, and that Minikube is installed. In this lecture, Minikube will provide the Kubernetes cluster, while Docker Desktop provides the Docker engine that Minikube can use as its driver. We will therefore use Minikube as the Kubernetes environment, not Docker Desktop's built-in Kubernetes. This keeps the workflow simple and consistent.

4 What We Will Build

The application will work as follows. The user opens a browser and sees a page with one text input. The user types something such as `hello kubernetes` and presses submit. The web app receives

the request and writes that value into a MongoDB collection. The page then refreshes and shows the current saved value.

The point of the example is that the web app depends on the database. Kubernetes will therefore need to run two applications together and allow them to communicate. This is the first important lesson: Kubernetes is not just for running one container. It is for coordinating multiple containers that together form an application.

5 Project Structure

Create a folder for the demo and use the following structure.

```
k8s-mongo-demo/  
  app.py  
  requirements.txt  
  Dockerfile  
  k8s/  
    mongo-secret.yaml  
    mongo-configmap.yaml  
    mongo-pvc.yaml  
    mongo-deployment.yaml  
    mongo-service.yaml  
    webapp-deployment.yaml  
    webapp-service.yaml
```

This structure separates application files from Kubernetes manifests. That is a good habit in MLOps because application code and infrastructure code should be organized clearly.

6 Step 1: Create the Simple Web App

Create the file `app.py`.

```
from flask import Flask, request  
from pymongo import MongoClient  
import os  
  
app = Flask(__name__)  
  
mongo_user = os.environ.get("MONGO_USERNAME", "root")  
mongo_password = os.environ.get("MONGO_PASSWORD", "example")  
mongo_host = os.environ.get("MONGO_HOST", "mongo-service")  
mongo_port = os.environ.get("MONGO_PORT", "27017")  
mongo_db = os.environ.get("MONGO_DB", "demo_db")  
  
mongo_uri = f"mongodb://{mongo_user}:{mongo_password}@{mongo_host}:{mongo_port}  
  {mongo_db}/"  
client = MongoClient(mongo_uri)  
db = client[mongo_db]  
collection = db["settings"]  
  
def get_current_value():  
    doc = collection.find_one({"_id": "current"})
```

```

    if doc:
        return doc.get("value", "")
    return ""

@app.route("/", methods=["GET"])
def index():
    current_value = get_current_value()
    return f"""
<html>
  <body>
    <h2>Simple Kubernetes + Mongo Demo</h2>
    <form action="/update" method="post">
      <label>Enter a value:</label>
      <input type="text" name="value" />
      <button type="submit">Save</button>
    </form>
    <p><strong>Current saved value:</strong> {current_value}</p>
  </body>
</html>
"""

@app.route("/update", methods=["POST"])
def update():
    new_value = request.form.get("value", "")
    collection.update_one(
        {"_id": "current"},
        {"$set": {"value": new_value}},
        upsert=True
    )
    return f"""
<html>
  <body>
    <p>Saved value: <strong>{new_value}</strong></p>
    <a href="/">Go back</a>
  </body>
</html>
"""

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

This application is intentionally minimal. It uses Flask for the web server and PyMongo to talk to MongoDB. The root route displays a form and also reads the current value from the database. The /update route receives the submitted field and writes it into MongoDB. Notice that the MongoDB connection information is read from environment variables rather than hard-coded. This is important because Kubernetes configuration should be supplied externally.

7 Step 2: Create the Requirements File

Create requirements.txt.

```

flask
pymongo

```

```
gunicorn
```

We use Flask for the web application, PyMongo for database communication, and Gunicorn as the production process that will run the app inside the container.

8 Step 3: Create the Dockerfile

Create Dockerfile.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

This Dockerfile starts from a lightweight Python image, sets a working directory, installs the dependencies, copies the application, exposes port 5000, and starts the app using Gunicorn. In MLOps terms, this is the same packaging step you would later use for a model inference API.

9 Step 4: Start Minikube with the Docker Driver

Open a terminal in the project folder and run the following commands.

```
minikube start --driver docker
kubectl get nodes
```

The first command starts a local Kubernetes cluster and tells Minikube to use Docker as the virtualization driver. On macOS with Docker Desktop running, this is a straightforward setup. Internally, this means Minikube launches a container that behaves like a Kubernetes node, and inside that container the full Kubernetes control plane and worker components are started. Even though everything is running on one physical machine, Kubernetes still behaves as if it is managing a real cluster.

The second command checks that the Kubernetes node is in the Ready state. The program kubectl is the command-line tool that communicates with the Kubernetes API server. When this command is executed, it asks the Kubernetes control plane to return a list of all nodes that are currently part of the cluster. In a local Minikube setup, there is usually only one node, which acts as both the control plane and the worker node. The output typically shows the node name, its status, its role, and the Kubernetes version. If the status is shown as Ready, it means the cluster started successfully and Kubernetes is able to schedule and run pods. If the status is not ready, then the cluster is not fully initialized and deployments will not work.

10 Step 5: Build the Web App Image Directly into Minikube

Build the image so that the Minikube cluster can use it without pushing to a remote registry.

```
minikube image build -t simple-webapp:v1 .
```

This command builds the Docker image inside Minikube's image environment. That is useful for local development because the cluster can immediately access the image. If you use a normal `docker build` outside this workflow, Kubernetes inside Minikube may not see the image.

11 Step 6: Create the Kubernetes Secret for Mongo Credentials

Create `k8s/mongo-secret.yaml`.

```
apiVersion: v1
kind: Secret
metadata:
  name: mongo-secret
type: Opaque
stringData:
  mongo-root-username: root
  mongo-root-password: example123
```

Now apply it.

```
kubectl apply -f k8s/mongo-secret.yaml
```

The first line, `apiVersion: v1`, tells Kubernetes which API group and version should be used to interpret this object. Secrets belong to the core Kubernetes API, so the version used here is `v1`. Every Kubernetes YAML file must specify an API version so that the Kubernetes API server knows how to validate and store the object.

The next line, `kind: Secret`, tells Kubernetes what type of object we are creating. Kubernetes supports many different object types such as `Pod`, `Deployment`, `Service`, `ConfigMap`, and `Secret`. By specifying `Secret`, we are telling Kubernetes that this file should create a secret object in the cluster. This means the values inside the file will be stored in Kubernetes' internal storage and can later be injected into containers as environment variables or mounted files.

The `metadata` section contains information that identifies the object. The field `name: mongo-secret` gives the secret a name inside the cluster. This name is important because other Kubernetes objects, such as a `Deployment`, will refer to this secret by name when they need to read the stored values. For example, the MongoDB container or the web application container will later use `mongo-secret` to obtain the username and password.

The line `type: Opaque` defines the type of secret. In Kubernetes, an opaque secret is the most general type and simply means that the secret contains arbitrary key-value pairs. Kubernetes also supports special secret types for things like TLS certificates or Docker registry credentials, but for simple username and password storage, `Opaque` is the correct and most common choice.

The `stringData` section contains the actual values that we want to store. Inside this section, we define keys and values. In this case, the key `mongo-root-username` has the value `root`, and the key `mongo-root-password` has the value `example123`. Kubernetes will store these values securely inside

the cluster. Later, when we create the MongoDB Deployment, we will reference these keys so that the container receives them as environment variables. Using `stringData` allows us to write the values in plain text, and Kubernetes will automatically convert them into the encoded format that secrets use internally.

The reason we create a secret instead of writing the username and password directly inside the Deployment YAML is that credentials should not be hard-coded into application configuration. In real systems, the same Deployment file might be used in different environments such as development, testing, and production, each with different passwords. By keeping the credentials in a Secret, we can change them without modifying the Deployment. This separation of configuration and code is an important best practice in Kubernetes and is commonly used in MLOps systems when storing database passwords, API tokens, or model registry credentials.

The word `apply` means that Kubernetes should create the object if it does not exist, or update it if it already exists. This is different from older commands like `create`, which only work if the object does not exist yet. The `apply` command is commonly used in real systems because it allows you to run the same command many times and Kubernetes will keep the cluster in the desired state described in the YAML file.

The option `-f` means “file”. It tells `kubectl` that the configuration should be read from a file. The path `k8s/mongo-secret.yaml` is the location of the YAML file that contains the Secret definition. When this command runs, `kubectl` reads the file, sends the contents to the Kubernetes API server, and the API server stores the Secret inside the cluster.

After running this command, the Secret named `mongo-secret` exists inside Kubernetes. It is not inside your local folder anymore; it is stored inside the Kubernetes cluster. Other Kubernetes objects, such as Deployments, can now reference this secret and use the stored values as environment variables.

The command to get all the secrets in kubernetes cluster is:

```
kubectl get secrets
```

12 Step 7: Create the ConfigMap for Non-Sensitive Mongo Settings

Create `k8s/mongo-configmap.yaml`.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mongo-config
data:
  mongo-database: demo_db
  mongo-host: mongo-service
  mongo-port: "27017"
```

Apply it.

```
kubectl apply -f k8s/mongo-configmap.yaml
```

`apiVersion: v1` tells Kubernetes to use the core API version for this object. `kind: ConfigMap` tells Kubernetes that this file creates a ConfigMap resource. These info are not confidential so they are

being stored in kubernetes configmap object. `metadata.name: mongo-config` gives the ConfigMap a name so other objects can reference it. The data section contains key-value pairs that will be passed to containers later. `mongo-database: demo_db` This defines the database name that the app should use. `mongo-host: mongo-service` This tells the web app to connect to MongoDB using the Kubernetes Service named `mongo-service`. In Kubernetes, services provide stable network names, so containers do not connect using IP addresses. As you can remember, nodes, pods go down all the time, new ones get created all the time. This is why IP is not used, instead the service names are used. `mongo-port: "27017"` This defines the MongoDB port. It is written as a string because ConfigMap values are stored as text.

In short, this ConfigMap stores database connection settings that the web app will read as environment variables instead of hard-coding them inside the application.

13 Step 8: Create Persistent Storage for MongoDB

Create `k8s/mongo-pvc.yaml`.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Apply it.

```
kubectl apply -f k8s/mongo-pvc.yaml
kubectl get pvc
```

This file defines a `PersistentVolumeClaim`, which is used to request storage from Kubernetes. MongoDB is a stateful application, meaning that it stores data that must survive container restarts. If the database files are stored only inside the container filesystem, the data will be lost whenever the Pod is deleted or recreated. A `PersistentVolumeClaim` allows the Pod to use external storage so that the data remains even if the container stops.

The field `apiVersion: v1` tells Kubernetes to use the core API version for this object, and `kind: PersistentVolumeClaim` tells Kubernetes that this file is requesting storage. The name `mongo-pvc` is used later in the MongoDB Deployment to attach this storage to the container.

Inside the `spec` section, the field `accessModes` describes how the storage can be used. The value `ReadWriteOnce` means that the volume can be mounted as read-write by one node at a time, which is sufficient for a single MongoDB instance in this demo. The `resources.requests.storage` field asks Kubernetes to allocate one gigabyte of storage. Kubernetes will provide a volume that satisfies this request.

After this `PersistentVolumeClaim` is created, the MongoDB Pod can mount this volume and store its database files there. If the Pod restarts, the data will remain in the volume instead of being lost with the container.

14 Step 9: Create the MongoDB Deployment

Create `k8s/mongo-deployment.yaml`.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
        - name: mongo
          image: mongo:latest
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: mongo-root-username
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: mongo-root-password
          volumeMounts:
            - name: mongo-storage
              mountPath: /data/db
      volumes:
        - name: mongo-storage
          persistentVolumeClaim:
            claimName: mongo-pvc
```

This file defines a Kubernetes Deployment that runs a MongoDB container. A Deployment is used to manage Pods automatically and ensure that the desired number of Pods is always running. In this case, we want one MongoDB instance running inside the cluster.

The field `apiVersion: apps/v1` is used because Deployment belongs to the apps API group. The field `kind: Deployment` tells Kubernetes that this object should manage Pods. The name `mongo-deployment` is the identifier for this Deployment inside the cluster.

Inside the `spec` section, the field `replicas: 1` means that only one MongoDB Pod should run. For a simple demo, one replica is enough. The `selector.matchLabels` field tells Kubernetes which Pods belong to this Deployment. The Deployment will manage Pods that have the label

`app: mongo`. The same label appears in the Pod template so that the Deployment can recognize the Pods it creates.

The `template` section defines the actual Pod that will be created. Inside the Pod specification, we define one container named `mongo` using the image `mongo:latest`. This is how the pod is connected to the deployment, because the deployment has match labels for all the pods named `app:mongo`. This tells Kubernetes to run the official MongoDB container from Docker Hub. The `containerPort: 27017` line documents the port used by MongoDB inside the container.

The `env` section defines environment variables that are passed into the container. Instead of writing the username and password directly in the file, the values are read from the Secret created earlier. The field `valueFrom.secretKeyRef` tells Kubernetes to read the value from the Secret named `mongo-secret`. The keys `mongo-root-username` and `mongo-root-password` are the values stored inside that Secret. This keeps credentials separate from the Deployment configuration.

The `volumeMounts` section attaches storage to the container. MongoDB stores its data in the directory `/data/db`, so the volume is mounted at that path. The `volumes` section connects this mount to the PersistentVolumeClaim created earlier. The field `claimName: mongo-pvc` tells Kubernetes to use the storage requested in the PVC. This ensures that the database data will remain even if the Pod restarts. So, you are mounting the pod's mongo's data/db to the pvc that is outside the pod.

After creating the file, apply it using the following commands.

```
kubectl apply -f k8s/mongo-deployment.yaml
kubectl get deployments
kubectl get pods
```

The first command sends the Deployment definition to Kubernetes. The second command shows the list of Deployments, and the third command shows the Pods that were created. If the Deployment is correct, one MongoDB Pod should appear in the Running state.

15 Step 10: Create the MongoDB Service

Create `k8s/mongo-service.yaml`.

```
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
spec:
  selector:
    app: mongo
  ports:
    - protocol: TCP
      port: 27017
      targetPort: 27017
  type: ClusterIP
```

Apply it.

```
kubectl apply -f k8s/mongo-service.yaml
kubectl get services
```

A Service in Kubernetes is a networking object that provides a stable way for Pods to communicate with each other. Pods are temporary and can be created or destroyed at any time, which means their IP addresses may change. If one application tried to connect directly to a Pod, the connection could break whenever the Pod restarts. A Service solves this problem by giving a fixed name and address that always points to the correct Pods. The Service uses labels to find matching Pods and automatically forwards traffic to them. In this example, the MongoDB Service allows other containers in the cluster to connect to the database using the name `mongo-service` instead of using a changing Pod IP. Because the Service type is `ClusterIP`, the database is only accessible inside the cluster, which is the correct design for internal components such as databases in web applications or MLOps systems.

The field `apiVersion: v1` indicates that this object belongs to the core Kubernetes API, and `kind: Service` tells Kubernetes that this resource should provide networking access to Pods. The name `mongo-service` is the identifier that other objects will use to connect to MongoDB. For example, the web application will connect to MongoDB using the hostname `mongo-service` instead of an IP address.

Inside the `spec` section, the `selector` field tells the Service which Pods it should connect to. In this case, the Service selects Pods with the label `app: mongo`. This matches the label defined in the MongoDB Deployment, so the Service will route traffic to the MongoDB Pod created by that Deployment.

The `ports` section defines how the Service forwards network traffic. The field `port: 27017` is the port exposed by the Service inside the cluster, and `targetPort: 27017` is the port used by the MongoDB container. The protocol is set to `TCP`, which is the standard protocol used by MongoDB.

The field `type: ClusterIP` means that this Service is only accessible from inside the Kubernetes cluster. This is the correct choice for a database, because the web application should be able to reach MongoDB, but the database should not be exposed directly to the outside world.

After creating the file, apply it with the following command.

```
kubectl apply -f k8s/mongo-service.yaml
kubectl get services
```

The first command creates the Service in the cluster, and the second command lists all Services. If the Service was created correctly, the name `mongo-service` will appear in the list. Other Pods in the cluster can now connect to MongoDB using the hostname `mongo-service` on port 27017.

16 Step 11: Create the Web App Deployment

Create `k8s/webapp-deployment.yaml`.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
```

```

    app: simple-webapp
template:
  metadata:
    labels:
      app: simple-webapp
  spec:
    containers:
      - name: simple-webapp
        image: simple-webapp:v1
        imagePullPolicy: IfNotPresent
        ports:
          - containerPort: 5000
        env:
          - name: MONGO_USERNAME
            valueFrom:
              secretKeyRef:
                name: mongo-secret
                key: mongo-root-username
          - name: MONGO_PASSWORD
            valueFrom:
              secretKeyRef:
                name: mongo-secret
                key: mongo-root-password
          - name: MONGO_DB
            valueFrom:
              configMapKeyRef:
                name: mongo-config
                key: mongo-database
          - name: MONGO_HOST
            valueFrom:
              configMapKeyRef:
                name: mongo-config
                key: mongo-host
          - name: MONGO_PORT
            valueFrom:
              configMapKeyRef:
                name: mongo-config
                key: mongo-port

```

Apply it.

```

kubectl apply -f k8s/webapp-deployment.yaml
kubectl get pods

```

This file defines a Deployment that runs the web application container inside Kubernetes. A Deployment is used to manage Pods automatically and ensure that the application is always running. In this case, the Deployment will start one Pod that runs the container image `simple-webapp:v1`, which was built earlier using Docker.

The field `apiVersion: apps/v1` is used because Deployment belongs to the apps API group. The field `kind: Deployment` tells Kubernetes that this object manages Pods. The name `webapp-deployment` is the identifier for this Deployment inside the cluster.

Inside the `spec` section, the field `replicas: 1` means that only one Pod of the web application should run. The `selector.matchLabels` field tells Kubernetes which Pods belong to this Deployment. The Pod template also contains the label `app: simple-webapp` so that the Deployment can recognize and manage the Pods it creates.

The container definition specifies the image `simple-webapp:v1`, which is the Docker image that contains the Flask application. The field `imagePullPolicy: IfNotPresent` tells Kubernetes to use the local image if it already exists, which is useful when running Minikube locally. The field `containerPort: 5000` indicates the port used by the web application inside the container.

The `env` section defines environment variables that are passed to the container. These values are not written directly in the Deployment. Instead, they are read from the Secret and ConfigMap created earlier. The MongoDB username and password are read from the Secret `mongo-secret`, while the database name, host, and port are read from the ConfigMap `mongo-config`. This keeps configuration separate from the application code and allows the same container image to be used in different environments.

After this Deployment is applied, Kubernetes will create a Pod running the web application, and the application will be able to connect to MongoDB using the Service name `mongo-service`. Make sure to use the `get pods` command to see if all the pods are running and ready.

17 Step 12: Create the Web App Service

Create `k8s/webapp-service.yaml`.

```
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
spec:
  selector:
    app: simple-webapp
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  type: NodePort
```

This file defines a Kubernetes Service that exposes the web application so that it can be accessed from outside the cluster. The web application is running inside a Pod created by the Deployment, but Pods do not automatically have a stable address and are not directly reachable from the host machine. A Service provides a permanent network endpoint that forwards traffic to the correct Pod.

The field `apiVersion: v1` indicates that this object belongs to the core Kubernetes API, and `kind: Service` tells Kubernetes that this resource should provide networking access to Pods. The name `webapp-service` is the identifier for this Service inside the cluster.

Inside the `spec` section, the `selector` field tells the Service which Pods it should connect to. In this case, the Service selects Pods that have the label `app: simple-webapp`. This matches the label defined in the web application Deployment, so the Service will route requests to the web application Pod.

The `ports` section defines how traffic is forwarded. The field `port: 5000` is the port exposed by the Service, and `targetPort: 5000` is the port used by the container inside the Pod. The protocol is TCP, which is the standard protocol for web applications.

The field `type: NodePort` means that the Service will be exposed outside the cluster through a port on the Kubernetes node. When using Minikube, this allows the web application to be accessed from the local machine using the command `minikube service webapp-service -url`. This type of Service is useful for local development because it makes the application reachable without needing a cloud load balancer.

After applying this file, the web application can be accessed through the Service, and the Service will automatically forward requests to the running Pod.

Apply it.

```
kubectl apply -f k8s/webapp-service.yaml
kubectl get services
```

The web app must be reachable from your laptop browser, so we use a NodePort service. The selector targets pods labeled `app: simple-webapp`. The service exposes port 5000 and forwards traffic to container port 5000. The `type: NodePort` tells Kubernetes to expose the application externally through the node.

18 Step 13: Access the Web App

Use Minikube to get the URL.

```
minikube service webapp-service --url
```

Copy the URL into your browser. You should see the form. Enter a value and submit it. The web app should save the value into MongoDB and show it back to you.

19 Step 14: Verify the Database Was Updated

To verify from the command line, first find the MongoDB pod.

```
kubectl get pods
```

Then open a shell inside the Mongo pod.

```
kubectl exec -it <mongo-pod-name> -- bash
```

Inside the pod, open the Mongo shell.

```
mongosh -u root -p example123 --authenticationDatabase admin
```

Then run:

```
use demo_db
db.settings.find()
```

You should see a document with `_id: "current"` and the saved value. This confirms that the web app wrote to MongoDB successfully.

20 Step 15: Debugging Commands

If something does not work, the first step is always to inspect the Kubernetes resources.

```
kubectl get all
kubectl get pods
kubectl get services
kubectl get deployments
kubectl describe pod <pod-name>
kubectl logs <pod-name>
```

If the web app cannot connect to MongoDB, check the web app logs. If MongoDB failed to start, check the MongoDB pod logs. If the page is not reachable, check the Service and make sure you are using the URL returned by `minikube service`. If the pods are stuck in an image error state, make sure the local image was built with `minikube image build`.

21 Cleanup

When the demo is over, remove the resources.

```
kubectl delete -f k8s/webapp-service.yaml
kubectl delete -f k8s/webapp-deployment.yaml
kubectl delete -f k8s/mongo-service.yaml
kubectl delete -f k8s/mongo-deployment.yaml
kubectl delete -f k8s/mongo-pvc.yaml
kubectl delete -f k8s/mongo-configmap.yaml
kubectl delete -f k8s/mongo-secret.yaml
```

To stop Minikube:

```
minikube stop
```

22 References

1. Kubernetes Documentation. Overview. <https://kubernetes.io/docs/concepts/>
2. Kubernetes Documentation. Deployments. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
3. Kubernetes Documentation. Services. <https://kubernetes.io/docs/concepts/services-network/service/>
4. Kubernetes Documentation. Secrets. <https://kubernetes.io/docs/concepts/configuration/secret/>
5. Kubernetes Documentation. ConfigMaps. <https://kubernetes.io/docs/concepts/configuration/configmap/>
6. Kubernetes Documentation. Persistent Volumes. <https://kubernetes.io/docs/concepts/storage/persistent-volumes/>
7. Kubernetes Documentation. Configure a Pod to Use a PersistentVolumeClaim for Storage. <https://kubernetes.io/docs/tasks/configure-pod-container/configure-persistent-volume-claim/>

8. Minikube Documentation. Start. <https://minikube.sigs.k8s.io/docs/commands/start/>
9. Minikube Documentation. Service. <https://minikube.sigs.k8s.io/docs/commands/service/>
10. Minikube Documentation. Accessing Apps. <https://minikube.sigs.k8s.io/docs/handbook/accessing/>
11. Docker Desktop Documentation. Kubernetes. <https://docs.docker.com/desktop/use-desktop/kubernetes/>
12. Mongo Official Image. Docker Hub. https://hub.docker.com/_/mongo